

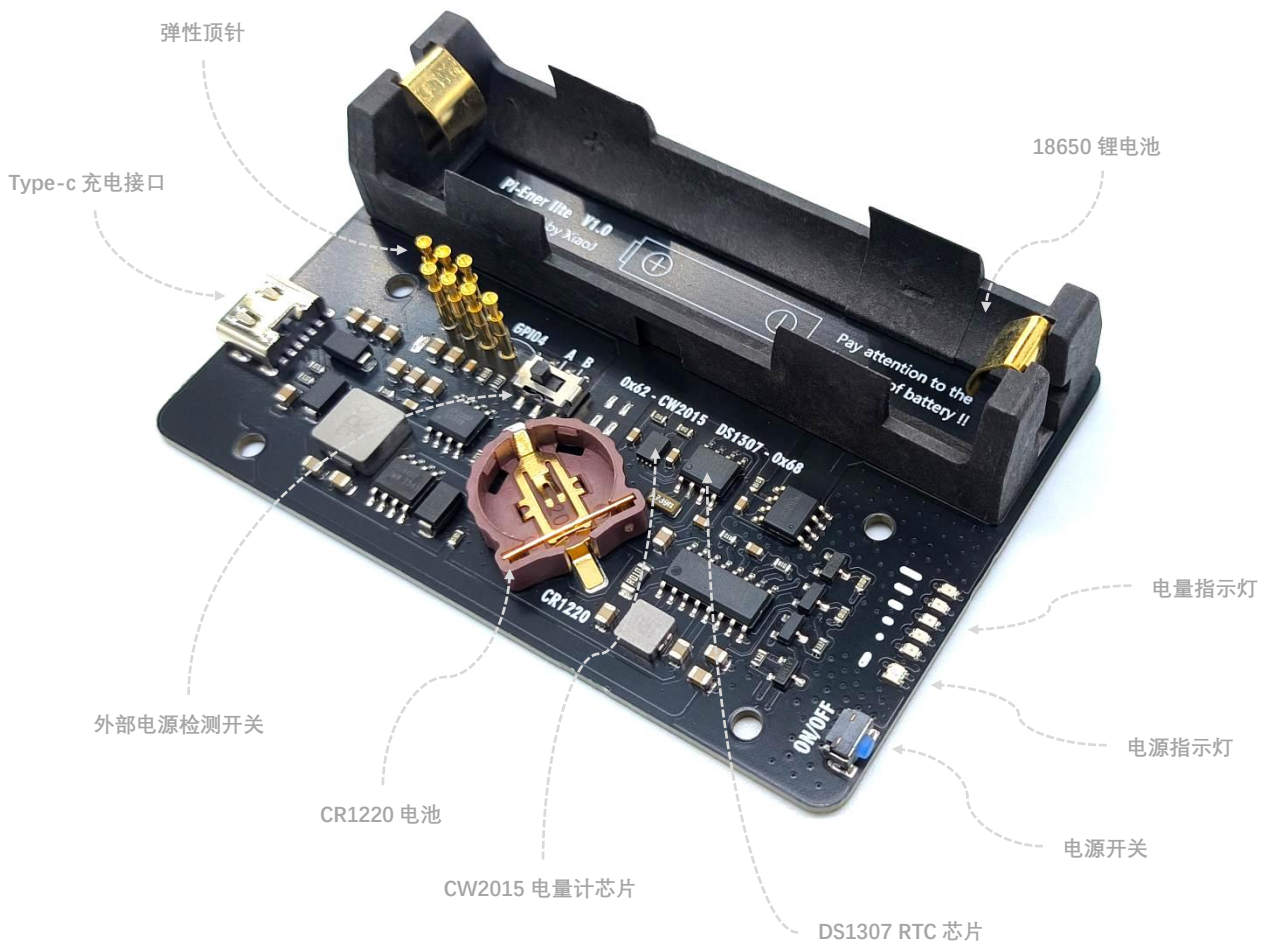
Pi-Ener-lite 使用说明

V1.0 -- by XiaoJ



1. 简介

Pi-Ener-lite 是一款专门为树莓派 Zero 系列主板设计的 UPS 电源，通过 7 根弹性顶针与 Zero 主板连接，供电以及数据读取都是通过顶针来完成。Pi-Ener-lite 采用标准的平头 18650 锂电池进行供电，外部电源断电时无缝切换为锂电池供电，支持边充边放，支持过充、过放、过流、短路保护，以及外部电源插入检测。另外 Pi-Ener-lite 集成了电量计芯片 CW2015 以及 RTC 实时时钟芯片 DS1307, 可用于对电池电压、电量进行测量, 以及为 Zero 提供 RTC 计时功能。



参数:

充电电流: **1A@5V**

放电电流: **2.5A@5.1V** (请使用质量好的电池以及电源适配器, 以确保 UPS 正常工作)

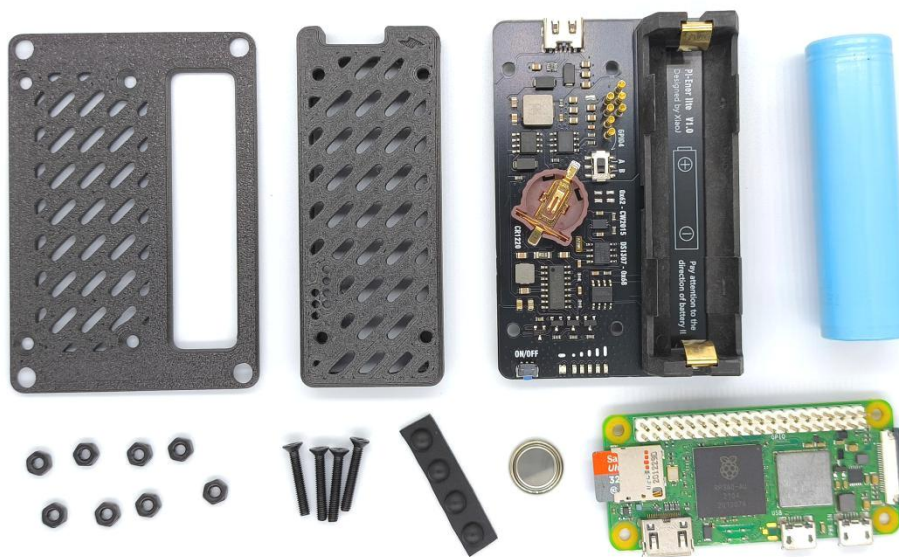
电池测量: 电压测量误差在 $\pm 3\text{mV}$, (测量误差因电池个体、环境温度而异)

电池剩余电量 SOC 百分比, 误差 $\pm 2\%$

2. 组装

请准备好以下材料, 以便进行安装

备注: 请自备 Zero 主板, 18650 电池, CR1220 纽扣电池



- a. 安装 CR1220 纽扣电池。把外部电源检测开关拨到 “A” 位置，使能外部电源插入检测功能



- b. 组装 3D 打印配件



c. 安装防滑硅胶脚垫



d. 安装 Zero 主板以及 18650 锂电池



3. 功能使用

3.1 电源开关操作及指示灯说明

长按电源开关 3 秒以上可以开启或者关闭电源输出。电源输出时，红色灯亮，电源关闭时，红色灯灭。

插入外部电源时，蓝色灯亮，拔出外部电源时，蓝色灯灭。

5 个电量指示灯表示的电池容量依次为 20%、40%、60%、80%、100%。未充电的情况下，短按电源开关时电量指示灯可以显示当前电池的大概电量，稍后电量指示灯会自动熄灭。充电过程中，电量指示灯会相继闪烁，当电池充满电时，5 个电量指示灯会全亮，表示电池已经充满。

3.2 外部电源插入检测功能

该功能是通过 Zero 读取 GPIO4 电平高低来判断外部电源是否插入，要使用该功能时必须把外部电源检测开关拨到“A”位置，这会把 GPIO4 接入到电平判断电路。GPIO4 电平为高时，则表示未插入电源，GPIO4 电平为低时，则表示已插入电源。

如果 GPIO4 有其他用途，可以把开关拨到“B”位置，断开与电平判断电路的连接。

3.3 RTC 实时时钟功能

Pi-Ener-lite 采用 DS1307 作为系统的 RTC 计时时钟。在使用之前必须确保 RTC 备用电池 CR1220 正确安装，否则系统断电后 RTC 芯片将停止计时。RTC 设置可参考以下步骤：

a. 打开 pi 配置工具 raspi-config，使能 I2C 接口



```
j@raspberrypi: ~  
j@raspberrypi:~ $ sudo raspi-config
```

b. 确保系统已经安装了 i2c-tools 工具包。如果没有安装 i2c-tools, 请使用以下命令安装

sudo apt install i2c-tools

使用 i2cdetect 命令扫描 i2c 设备, 其中地址 0x62 是 CW2015, 地址 0x68 是 DS1307

sudo i2cdetect -y 1

```
j@raspberrypi:~$ sudo i2cdetect -y 1
   0  1  2  3  4  5  6  7  8  9  a  b  c  d  e  f
00: -- -- -- -- -- -- -- -- -- -- -- -- -- -- --
10: -- -- -- -- -- -- -- -- -- -- -- -- -- -- --
20: -- -- -- -- -- -- -- -- -- -- -- -- -- -- --
30: -- -- -- -- -- -- -- -- -- -- -- -- -- -- --
40: -- -- -- -- -- -- -- -- -- -- -- -- -- -- --
50: -- -- -- -- -- -- -- -- -- -- -- -- -- -- --
60: -- -- 62 -- -- -- -- -- 68 -- -- -- -- -- --
70: -- -- -- -- -- -- -- -- -- -- -- -- -- -- --

j@raspberrypi:~$
```

CW2015 DS1307

c. 使用 nano 编辑器修改系统配置文件 config.txt

sudo nano /boot/firmware/config.txt

在 config.txt 末端添加以下文本, 加载 DS1307 设备树

dtoverlay=i2c-rtc,ds1307

```
GNU nano 8.4 /boot/firmware/config.txt *
arm_boost=1

[cm4]
# Enable host mode on the 2711 built-in XHCI USB controller.
# This line should be removed if the legacy DWC2 controller is required
# (e.g. for USB device mode) or if USB support is not required.
otg_mode=1

[cm5]
dtoverlay=dwc2,dr_mode=host

[all]
enable_uart=1

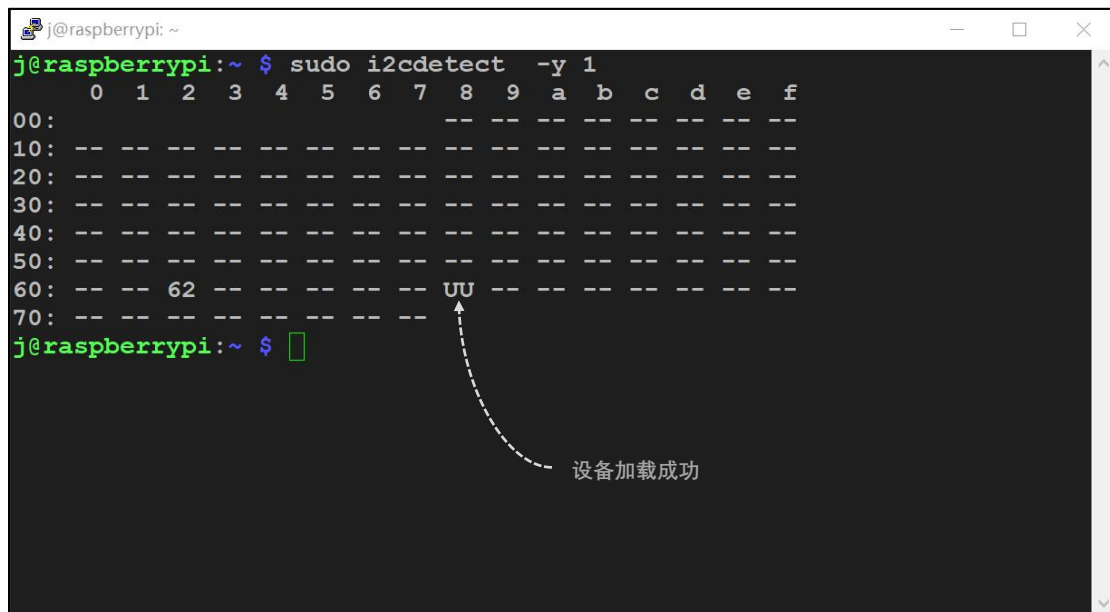
dtoverlay=i2c-rtc,ds1307

```

添加在这里

d. 重启系统，再次使用 i2cdetect 命令扫描，如果 0x68 地址变为 0xUU 则表示设备加载成功

sudo i2cdetect -y 1

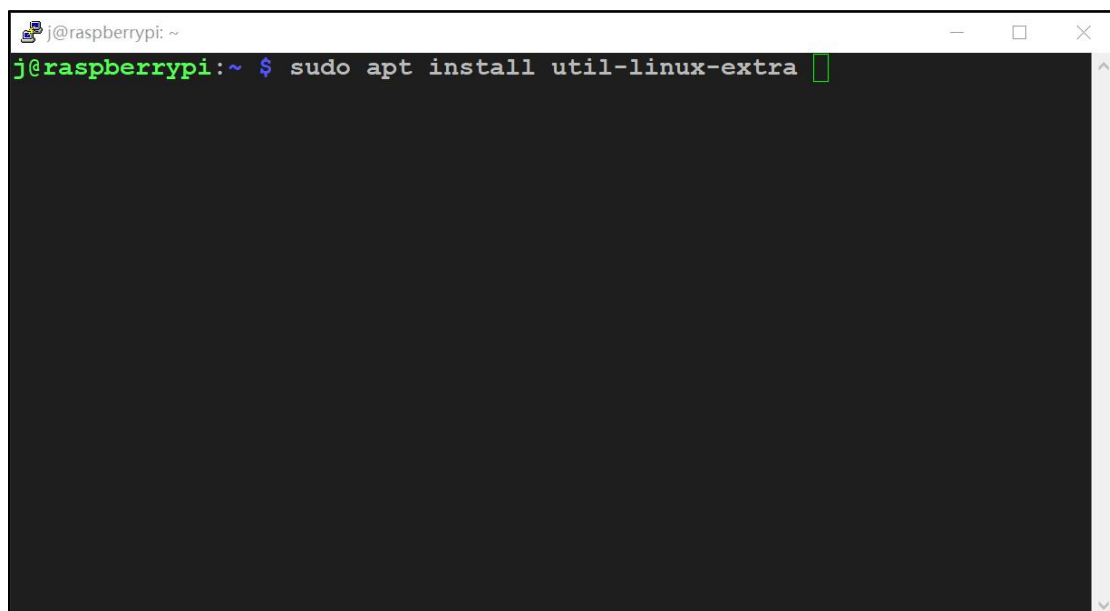


```
j@raspberrypi: ~  
j@raspberrypi:~$ sudo i2cdetect -y 1  
    0  1  2  3  4  5  6  7  8  9  a  b  c  d  e  f  
00:  --  --  --  --  --  --  --  --  --  --  --  --  --  --  --  
10:  --  --  --  --  --  --  --  --  --  --  --  --  --  --  --  
20:  --  --  --  --  --  --  --  --  --  --  --  --  --  --  --  
30:  --  --  --  --  --  --  --  --  --  --  --  --  --  --  --  
40:  --  --  --  --  --  --  --  --  --  --  --  --  --  --  --  
50:  --  --  --  --  --  --  --  --  --  --  --  --  --  --  --  
60:  --  --  62  --  --  --  --  --  UU  --  --  --  --  --  --  
70:  --  --  --  --  --  --  --  --  --  --  --  --  --  --  --  
j@raspberrypi:~$
```

设备加载成功

e. 安装 util-linux-extra 工具包

sudo apt install util-linux-extra



```
j@raspberrypi: ~  
j@raspberrypi:~$ sudo apt install util-linux-extra
```

f. 调用 hwclock 命令设置 RTC 计时芯片，更多关于 hwclock 命令的设置请自行查阅

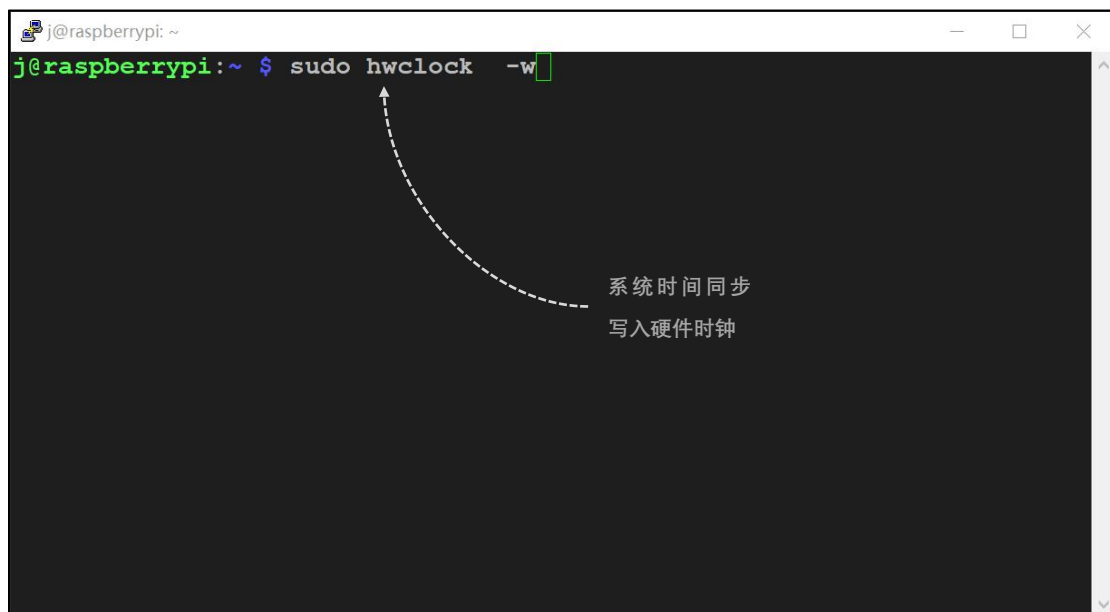
硬件时钟读取命令： *sudo hwclock -r*

系统时间同步写入硬件时钟命令： *sudo hwclock -w*



```
j@raspberrypi: ~  
j@raspberrypi:~$ sudo hwclock -r  
2026-08-13 22:45:18.300042+08:00  
j@raspberrypi:~$
```

读取硬件时钟



```
j@raspberrypi: ~  
j@raspberrypi:~$ sudo hwclock -w
```

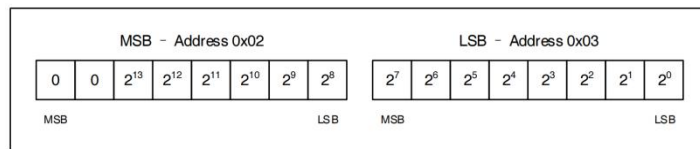
系统时间同步
写入硬件时钟

3.4 电池计量功能

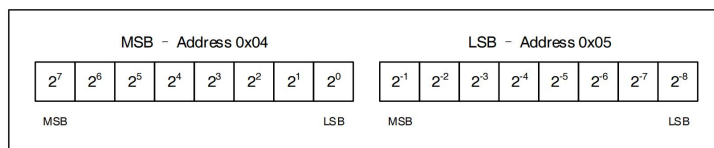
Pi-Ener-lite 采用 CW2015 作为电池电压、电量的测试方案，Zero 可以通过 I2C 接口与该芯片进行通信，读取电池电压、电量测量值等。

CW2015 设备地址为 0x62。寄存器 VCELL 为 14bit 电池电压 ADC 测量值，地址为 02h-03h，ADC 测量精度单位为 305uV。寄存器 SOC 为 16bit 电池容量百分比读数，地址为 04h-05h，SOC 的高 8bit 单位为电池容量的 1%，低 8bit 单位为 1/256%，提供电池容量百分比小数位的读数。通过对这两个寄存器数据的读取计算后便可得到当前电池电压及电量值。具体计算过程可参考提供的 Python 程序。关于 CW2015 更多的信息，请仔细阅读官方芯片手册。

VCELL 寄存器



SOC 寄存器



运行参考程序，读取电池电压、电量、外部电源插入情况

```
j@raspberrypi: /home/Pi-Ener-Lite
j@raspberrypi: /home/Pi-Ener-Lite $ sudo python Pi-Ener-Lite.py

Initialize the CW2015 .....
+++++++
Voltage: 4.17V
Battery: 100%
Battery FULL
Power Adapter Plug In
+++++++
+++++++
Voltage: 4.17V
Battery: 100%
Battery FULL
Power Adapter Plug In
+++++++
+++++++
Voltage: 4.17V
Battery: 100%
Battery FULL
Power Adapter Plug In
+++++++
+++++++
Voltage: 4.17V
Battery: 100%
Battery FULL
Power Adapter Plug In
+++++++
```

Pi-Ener-lite.py 参考程序:

```
#!/usr/bin/env python
import struct
import smbus
import sys
import time
import RPi.GPIO as GPIO

CW2015_ADDRESS    = 0X62
CW2015_REG_VCELL  = 0X02
CW2015_REG_SOC    = 0X04
CW2015_REG_MODE   = 0X0A

def readVoltage(bus):
    "This function returns as float the voltage from the Raspi UPS Hat via
    the provided SMBus object"
    read = bus.read_word_data(CW2015_ADDRESS, CW2015_REG_VCELL)
    swapped = struct.unpack("<H", struct.pack(">H", read))[0]
    voltage = swapped * 0.305 / 1000
    return voltage

def readCapacity(bus):
    "This function returns as a float the remaining capacity of the battery
    connected to the Raspi UPS Hat via the provided SMBus object"
    read = bus.read_word_data(CW2015_ADDRESS, CW2015_REG_SOC)
    swapped = struct.unpack("<H", struct.pack(">H", read))[0]
    capacity = swapped / 256
    return capacity

def QuickStart(bus):
    "This function wake up the CW2015 and make a quick-start fuel-gauge
    calculations "
    bus.write_word_data(CW2015_ADDRESS, CW2015_REG_MODE, 0x30)
```

```

#-----
GPIO.setmode(GPIO.BCM)
GPIO.setwarnings(False)
GPIO.setup(4,GPIO.IN) # GPIO4 is used to detect whether an external power supply
is inserted

bus = smbus.SMBus(1) # 0 = /dev/i2c-0 (port I2C0), 1 = /dev/i2c-1 (port I2C1)

QuickStart(bus)

print (" ")
print ("Initialize the CW2015 .....")

while True:

    print ("+++++")
    print ("Voltage:%5.2fV" % readVoltage(bus))
    print ("Battery:%5i%" % readCapacity(bus))

    if readCapacity(bus) == 100:
        print ("Battery FULL")
    if readCapacity(bus) < 5:
        print ("Battery LOW")

#GPIO is low when power is plugged in
if (GPIO.input(4) == GPIO.LOW):
    print ("Power Adapter Plug In ")
if (GPIO.input(4) == GPIO.HIGH):
    print ("Power Adapter Unplug")

print ("+++++")
time.sleep(2)

```

4. 其他

本项目不定期更新使用说明、参考程序、外壳设计等文件，最新资料请以 GitHub 上的为准

<https://github.com/linshuqin329/Pi-Ener-lite>

使用过程中有问题的话可以联系我: 416386001@qq.com